



Technologie Blockchain

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction : principes fondamentaux de la blockchain

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Définition

Une **blockchain** (chaîne de blocs) est un **registre distribué** (*distributed ledger*) : une base de données qui, au lieu d'être hébergée par une seule organisation sur un serveur central, est **répliquée intégralement** sur de nombreux nœuds indépendants d'un réseau pair-à-pair (*peer-to-peer*). Chaque nœud conserve sa propre copie complète (ou partielle, selon son rôle) du registre, et applique les mêmes règles de validation que les autres pour décider si une nouvelle donnée peut y être ajoutée.

Le registre est organisé en **blocs** : des paquets de données (typiquement des transactions) regroupés et validés ensemble, puis **chaînés cryptographiquement** les uns aux autres — chaque bloc référence le bloc précédent par son empreinte de hachage (détail complet au chapitre 2). Cette construction rend le contenu déjà validé **extrêmement difficile à modifier** a posteriori, sans qu'aucune autorité centrale unique n'ait besoin d'arbitrer la validité des opérations : la confiance repose sur des mécanismes cryptographiques et algorithmiques partagés par l'ensemble du réseau, et non sur la réputation d'un intermédiaire.

Il est utile de contraster cette approche avec une base de données classique : dans un système centralisé, un opérateur unique (une banque, un État, une entreprise) détient l'autorité finale sur ce qui est vrai dans le registre et peut, en théorie, modifier ou effacer un enregistrement. Une blockchain déplace cette autorité vers un **consensus collectif** entre participants qui ne se font pas nécessairement confiance individuellement, mais qui font tous confiance au protocole lui-même.

2. Genèse : Bitcoin (2008-2009)

En octobre 2008, un document intitulé « *Bitcoin: A Peer-to-Peer Electronic Cash System* » est publié sous le pseudonyme **Satoshi Nakamoto** (l'identité réelle de l'auteur ou du groupe d'auteurs n'a jamais été confirmée publiquement). Le réseau Bitcoin démarre effectivement en janvier 2009 avec la création du premier bloc, dit **bloc de genèse**.

L'innovation de Bitcoin ne réside pas dans l'invention de nouvelles primitives cryptographiques : les **signatures numériques** et les **fonctions de hachage** qu'il utilise étaient déjà connues et étudiées (module *Cryptographie*). Sa contribution originale est d'avoir combiné ces briques existantes avec un **mécanisme de consensus décentralisé** — la preuve de travail, détaillée au chapitre 3 — pour résoudre un problème resté ouvert jusque-là : le **problème de la double dépense** (*double-spending problem*).

Concrètement, ce problème peut se formuler ainsi : dans un système purement numérique, une donnée peut être copiée à l'infini sans coût. Comment empêcher qu'une même unité de monnaie numérique soit dépensée deux fois par son détenteur, en l'absence d'une banque centrale qui tiendrait les comptes et validerait chaque opération ? Les tentatives antérieures de monnaie numérique reposaient généralement sur un tiers de confiance central pour trancher ce type de conflit. Bitcoin propose à la place que ce soit **le réseau tout entier**, par le biais d'un mécanisme de consensus vérifiable par tous, qui détermine quelle transaction est valide et dans quel ordre elle a été émise.

3. Propriétés recherchées

La conception d'une blockchain vise typiquement un ensemble de propriétés qui, prises ensemble, permettent de se passer d'un tiers de confiance central. Le tableau suivant résume les quatre propriétés les plus souvent citées, avec leur signification précise :

Propriété	Signification
Décentralisation	aucune entité unique ne contrôle le registre ; il est répliqué et validé par de nombreux participants indépendants
Immutabilité (pratique)	modifier un bloc déjà validé et suffisamment confirmé nécessiterait de refaire le travail de validation de tous les blocs suivants, un coût prohibitif au-delà d'une certaine profondeur
Transparence	(pour les blockchains publiques) l'historique complet des transactions est consultable par tous
Résistance à la censure	aucune autorité unique ne peut empêcher une transaction valide d'être incluse, dans un réseau suffisamment décentralisé

La **décentralisation** n'est pas une propriété binaire : elle se mesure en degré, selon le nombre de participants réellement indépendants qui contribuent à la validation, la répartition géographique et organisationnelle de ces participants, et la difficulté pour un petit groupe de coordonner une majorité. Un réseau techniquement « décentralisé » sur le papier mais dont la puissance de validation est concentrée entre quelques acteurs perd une grande partie de l'intérêt du modèle.

L'**immutabilité** est qualifiée de « pratique » et non d'absolue à dessein : rien n'empêche techniquement de réécrire l'historique d'une blockchain, mais le coût de cette opération croît avec le nombre de blocs déjà ajoutés après celui que l'on souhaite modifier (voir chapitre 3, notion de confirmations). C'est un coût économique et calculatoire dissuasif, pas une impossibilité mathématique.

Il est essentiel de retenir que ces quatre propriétés sont des **objectifs de conception**, pas des garanties automatiques offertes par le simple fait d'utiliser une architecture en blocs chaînés. Elles dépendent fortement du degré réel de décentralisation du réseau considéré (chapitre 3) et de la robustesse des implémentations logicielles déployées (chapitre 5) : une blockchain mal conçue ou insuffisamment décentralisée peut échouer à fournir l'une ou l'autre de ces propriétés malgré les apparences.

4. Blockchain publique, privée, à permission

Toutes les blockchains ne partagent pas le même degré d'ouverture. On distingue généralement trois grandes catégories selon qui peut lire le registre et qui peut y écrire (proposer et valider de nouveaux blocs) :

Type	Accès en écriture	Accès en lecture	Exemple
Publique (permissionless)	ouvert à tous	ouvert à tous	Bitcoin, Ethereum
À permission (consortium)	limité à des participants identifiés	selon règles définies	réseaux inter-bancaires, chaînes logistiques inter-entreprises
Privée	contrôlé par une seule organisation	souvent restreint	usage interne d'entreprise, proche d'une base de données répliquée classique

Une **blockchain publique** (aussi dite *permissionless*) n'impose aucune condition d'accès : n'importe qui peut télécharger le logiciel client, rejoindre le réseau, envoyer des transactions et, selon le mécanisme de consensus, participer à la validation. C'est le modèle historique de Bitcoin et d'Ethereum, et celui qui offre les propriétés de décentralisation et de résistance à la censure les plus fortes.

Une **blockchain à permission (consortium)** restreint le droit d'écriture — et parfois de lecture — à un ensemble de participants identifiés et approuvés à l'avance, typiquement un groupe d'organisations qui se connaissent (banques d'un même consortium, entreprises d'une même chaîne logistique). Ce modèle sacrifie une partie de l'ouverture au profit d'une gouvernance plus simple et de performances souvent meilleures, puisque le nombre de validateurs est limité et connu.

Une **blockchain privée** pousse cette logique encore plus loin : une seule organisation contrôle entièrement le registre, décide qui peut y accéder et peut, en théorie, modifier les règles à sa guise. Dans ce cas, la structure en blocs chaînés apporte surtout des garanties d'intégrité et d'auditabilité interne, mais le résultat se rapproche fonctionnellement d'une base de données répliquée et journalisée classique — sans les propriétés de décentralisation qui motivent l'usage d'une blockchain publique.

Une confusion fréquente mérite d'être soulignée : une blockchain « privée » ou « à permission » perd une grande partie des propriétés de décentralisation et de résistance à la censure d'une blockchain publique. Le choix du type de réseau doit donc être justifié par le besoin réel du projet (a-t-on vraiment besoin qu'un tiers non identifié puisse rejoindre le réseau et y écrire ?), et non adopté par simple effet de mode ou par méconnaissance des compromis impliqués.

5. Anatomie d'un bloc (aperçu)

Un bloc se compose généralement de deux parties distinctes : un **en-tête** (*header*), de taille fixe et contenant les métadonnées nécessaires à la validation et au chaînage, et un **corps**, contenant la liste effective des transactions incluses dans ce bloc.

L'en-tête contient typiquement :

- l'**empreinte de hachage du bloc précédent**, qui matérialise le chaînage (détaillé au chapitre 2) ;
- un **horodatage** (*timestamp*), indiquant approximativement le moment de création du bloc ;
- la **racine de Merkle** des transactions du bloc, une empreinte unique résumant l'ensemble des transactions incluses (chapitre 2) ;
- un **nonce**, une valeur numérique ajustée par le mécanisme de consensus (en preuve de travail, notamment) pour satisfaire une condition de difficulté (chapitre 3).

Cette séparation entre en-tête compact et corps volumineux a une conséquence pratique importante, développée au chapitre suivant : un client « léger » peut se contenter de télécharger et vérifier les en-têtes des blocs, beaucoup plus légers que le corps complet, tout en conservant la capacité de prouver qu'une transaction donnée appartient bien à un bloc particulier grâce à la racine de Merkle.

6. Ce que la blockchain résout, et ce qu'elle ne résout pas

Il est tentant de présenter la blockchain comme une solution universelle à tout problème de confiance numérique. Un regard plus rigoureux impose de distinguer précisément ce qu'elle apporte réellement de ce qu'elle ne peut pas garantir par construction.

Elle résout la **cohérence d'un registre partagé sans tiers de confiance unique**, dans un contexte où les participants peuvent être mutuellement méfiants : une fois qu'une donnée est validée et suffisamment confirmée, tous les nœuds honnêtes du réseau s'accordent sur son contenu et son ordre, sans qu'aucun d'eux n'ait eu à faire confiance individuellement à un autre.

Elle ne résout en revanche pas, par elle-même, la **véracité des données saisies** — un principe souvent résumé par l'expression « *garbage in, garbage out* » (« à données d'entrée erronées, résultat erroné »). Une blockchain garantit l'**intégrité** de ce qui y a été inscrit : personne ne peut modifier discrètement une donnée après coup. Mais elle ne garantit en rien la **véracité** de cette information au moment de sa saisie initiale. Si une information fausse est inscrite sur la blockchain, elle y restera fidèlement conservée, fausse mais inaltérée. Ce point, connu sous le nom de « problème de l'oracle », sera approfondi au chapitre 4 à propos des smart contracts.

Enfin, l'usage d'une blockchain a un **coût réel**, souvent sous-estimé dans les discours promotionnels : le stockage est répliqué sur de nombreux nœuds (donc multiplié, plutôt que centralisé et optimisé) ; certains mécanismes de consensus impliquent une consommation énergétique significative (chapitre 3) ; la confirmation d'une transaction prend un temps non négligeable comparé à une mise à jour de base de données classique ; et l'exploitation opérationnelle d'un tel système (gestion de clés, gouvernance du réseau, tolérance aux pannes) ajoute une complexité réelle. Ces coûts imposent d'évaluer sérieusement, pour chaque projet envisagé, si une base de données traditionnelle avec un tiers de confiance ne répondrait pas mieux et plus simplement au besoin — question qui sera reprise en détail au chapitre 6.

À retenir

- La blockchain combine des primitives cryptographiques connues avec un mécanisme de consensus décentralisé pour résoudre le problème de la double dépense sans autorité centrale.
- Les propriétés de décentralisation, immutabilité et résistance à la censure sont des objectifs de conception dont le degré réel dépend de l'architecture choisie, pas des garanties automatiques.
- Les types publique, à permission et privée offrent des compromis très différents entre ouverture, gouvernance et performance.
- Une blockchain n'est pas une solution universelle : son usage doit être justifié par un besoin réel de décentralisation, pas adopté par principe.

Chapitre 2 — Structures de données : chaînage cryptographique et arbres de Merkle

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Le chaînage cryptographique des blocs

Le mécanisme qui donne son nom à la « chaîne de blocs » repose sur une idée simple mais puissante : chaque bloc contient, dans son en-tête, **l'empreinte de hachage du bloc précédent**. Une fonction de hachage cryptographique (module *Cryptographie*, chapitre 3) transforme une donnée d'entrée de taille quelconque en une empreinte de taille fixe, de façon déterministe, et présente une propriété essentielle ici : la moindre modification de l'entrée produit une empreinte totalement différente et imprévisible (effet d'avalanche).

Cette construction crée une **dépendance en cascade** entre tous les blocs de la chaîne : modifier le contenu d'un bloc ancien — ne serait-ce qu'un seul bit d'une seule transaction — change son empreinte de hachage. Or cette empreinte est précisément la valeur enregistrée dans l'en-tête du bloc suivant. L'empreinte enregistrée devient donc incorrecte, ce qui invalide le bloc suivant, ce qui invalide à son tour l'empreinte attendue par le bloc d'après, et ainsi de suite jusqu'au sommet de la chaîne.

Un attaquant souhaitant altérer un bloc ancien ne peut donc pas se contenter de modifier ce seul bloc : il doit recalculer l'intégralité des blocs qui le suivent, et — selon le mécanisme de consensus utilisé (chapitre 3) — refaire valider chacun d'eux plus rapidement que le reste du réseau honnête n'ajoute de nouveaux blocs légitimes. C'est un travail dont le coût croît directement avec la profondeur du bloc à modifier : plus un bloc est ancien et « enfoui » sous des blocs ultérieurs, plus le réécrire est coûteux. C'est ce raisonnement précis qui justifie l'expression d'**immuabilité « pratique »** plutôt qu'absolue introduite au chapitre 1, et qui motive la recommandation opérationnelle d'attendre un nombre suffisant de **confirmations** (c'est-à-dire de blocs ajoutés après celui contenant une transaction donnée) avant de considérer cette transaction comme définitivement acquise.

Un arbre de Merkle organise un ensemble de données — ici, les transactions d'un bloc — de façon à pouvoir **résumer l'ensemble par une seule empreinte** (la **racine de Merkle**, *Merkle root*), tout en conservant la possibilité de prouver efficacement qu'une donnée particulière fait bien partie de l'ensemble, sans avoir à transmettre l'ensemble complet des données.

Construction

La construction d'un arbre de Merkle procède par niveaux successifs, des feuilles vers la racine :

1. Chaque transaction est hachée individuellement ; ces empreintes forment les **feuilles** de l'arbre.
2. Les empreintes sont regroupées **deux par deux** et concaténées puis hachées ensemble, produisant une empreinte pour chaque paire : c'est le niveau immédiatement supérieur de l'arbre (s'il y a un nombre impair d'empreintes à un niveau donné, la dernière est généralement dupliquée pour compléter la paire, selon la convention retenue par l'implémentation).
3. Ce processus de regroupement et de hachage par paires est répété niveau par niveau, jusqu'à obtenir une seule et unique empreinte au sommet : la **racine de Merkle**, la seule valeur effectivement stockée dans l'en-tête du bloc (voir chapitre 1).

Ainsi, un bloc contenant des milliers de transactions peut être résumé par une empreinte de taille fixe (typiquement 256 bits pour une fonction comme SHA-256), tout en garantissant que toute modification, même minime, d'une seule transaction changerait la racine de Merkle du bloc — et donc, par le chaînage vu à la section précédente, invaliderait tout l'historique ultérieur.

Preuve de Merkle (Merkle proof)

L'intérêt pratique majeur de cette structure arborescente est qu'elle permet de prouver l'appartenance d'un élément **sans révéler ni transmettre l'ensemble des autres éléments**. Pour prouver qu'une transaction **T** appartient effectivement à un bloc donné, il suffit de fournir :

- la transaction **T** elle-même (ou son empreinte) ;
- la liste des empreintes « sœurs » situées sur le chemin remontant des feuilles jusqu'à la racine (une seule empreinte sœur par niveau).

Le vérificateur recalcule alors, niveau par niveau, l'empreinte du chemin en combinant **T** avec chacune des empreintes sœurs fournies, et compare le résultat final avec la racine de Merkle déjà connue (car incluse dans l'en-tête du bloc, lui-même chaîné et vérifiable). Si les deux valeurs correspondent, l'appartenance de **T** au bloc est prouvée. Le nombre d'empreintes à fournir est de l'ordre de **$\log_2(n)$** pour un bloc contenant n transactions — pour mille transactions, une dizaine d'empreintes suffisent, contre les mille transactions complètes qu'il aurait fallu transmettre sans cette structure.

3. Pourquoi cette double structure (chaîne de blocs + arbre par bloc) ?

Les deux mécanismes présentés jusqu'ici jouent des rôles complémentaires et ne se substituent pas l'un à l'autre :

- Le **chaînage entre blocs** garantit l'intégrité de l'**ordre et de l'historique global** : il rend coûteuse toute tentative de réorganisation ou de suppression rétroactive d'un bloc entier, et donc de la séquence des événements enregistrés.
- L'**arbre de Merkle au sein d'un bloc** garantit l'intégrité de l'**ensemble des transactions d'un bloc donné**, tout en permettant des vérifications partielles efficaces sans dépendre du volume total de données du bloc.

Ensemble, ces deux structures forment un système où toute modification, à quelque niveau que ce soit — une transaction individuelle ou l'ordre des blocs — se propage inévitablement jusqu'à une empreinte vérifiable par n'importe quel participant du réseau. Cette architecture illustre de façon directe comment des primitives cryptographiques relativement simples (fonctions de hachage, étudiées dans le module *Cryptographie*) peuvent servir de briques de construction pour obtenir des propriétés de bien plus haut niveau — ici, l'intégrité vérifiable d'un registre distribué entre des participants qui ne se font pas mutuellement confiance.

4. Identifiants et adresses

Le chaînage cryptographique et les arbres de Merkle protègent l'intégrité des données, mais un système de registre distribué a également besoin d'identifier de façon unique ses transactions et ses participants.

Une **transaction** est identifiée par l'empreinte de son propre contenu (le « hash de transaction », ou *transaction ID*, *TXID*) : puisque cette empreinte dépend intégralement du contenu de la transaction, deux transactions au contenu strictement identique produiraient la même empreinte, et la moindre différence de contenu produit un identifiant totalement différent — un identifiant qui sert donc à la fois de nom unique et de garantie d'intégrité.

Une **adresse** (représentant un compte ou un destinataire) dérive généralement d'une **clé publique**, elle-même souvent hachée à son tour pour produire une adresse plus courte et plus pratique à manipuler. Ce point relie directement ce chapitre au module *Cryptographie* (signatures numériques, chapitre 5) : chaque transaction sortante est **signée** par la clé privée correspondant à l'adresse d'origine, et cette signature est **vérifiable publiquement** par tout nœud du réseau à l'aide de la clé publique correspondante, sans que la clé privée n'ait jamais besoin d'être révélée. C'est ce mécanisme, et non un mot de passe ou une identité déclarée, qui prouve qu'une transaction a bien été autorisée par le détenteur légitime des fonds ou des droits concernés.

5. Variantes et structures alternatives

Le modèle « chaîne linéaire de blocs contenant chacun un arbre de Merkle » n'est pas la seule architecture possible pour un registre distribué sécurisé cryptographiquement. Deux variantes méritent d'être mentionnées pour situer ce modèle dans un paysage plus large :

- **DAG (Directed Acyclic Graph, graphe orienté acyclique)** : certaines architectures, comme celle du projet IOTA, remplacent la chaîne linéaire de blocs par un **graphe** de transactions individuelles, chacune référençant directement une ou plusieurs transactions précédentes plutôt que d'être regroupée en blocs successifs. L'objectif recherché est généralement une meilleure scalabilité (traitement parallèle plus naturel des transactions), au prix d'autres compromis en matière de sécurité, de maturité du protocole et de retour d'expérience en conditions réelles, comparés à des architectures en blocs plus anciennes et plus éprouvées.
- **État global (state trie)** : les blockchains supportant des smart contracts, au premier rang desquelles Ethereum (chapitre 4), ne se contentent pas de chaîner un historique de transactions : elles maintiennent également une **structure arborescente d'état** représentant la situation courante de tous les comptes et contrats du réseau (soldes, variables internes des contrats). Cette structure d'état est elle-même authentifiée cryptographiquement, avec une racine incluse dans chaque bloc — un principe directement analogue à celui de la racine de Merkle des transactions, mais appliqué cette fois à l'état courant du système plutôt qu'à son historique.

À retenir

- Le chaînage cryptographique entre blocs et l'arbre de Merkle au sein d'un bloc sont deux mécanismes complémentaires garantissant respectivement l'intégrité de l'historique global et celle du contenu de chaque bloc.
- Une preuve de Merkle permet de vérifier l'inclusion d'une transaction en transmettant seulement de l'ordre de $\log_2(n)$ empreintes, fondement des clients légers (SPV).
- Les transactions sont identifiées par leur propre empreinte et les adresses dérivent de clés publiques signées par la clé privée correspondante.
- Toute cette architecture repose directement sur les fonctions de hachage et les signatures numériques étudiées dans le module *Cryptographie*.

Chapitre 3 — Mécanismes de consensus

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Pourquoi un mécanisme de consensus ?

Le chapitre précédent a montré comment le chaînage cryptographique et les arbres de Merkle protègent l'**intégrité** d'un bloc et de l'historique une fois qu'ils sont acceptés par le réseau. Il reste cependant une question distincte, et tout aussi cruciale : **qui a le droit de décider quel bloc est ajouté ensuite**, et comment s'assurer que ce choix est accepté de la même manière par l'ensemble des participants ?

Dans un réseau décentralisé sans autorité centrale, potentiellement composé de milliers de participants anonymes ou pseudonymes, dont certains peuvent être malveillants ou simplement défaillants (panne réseau, matériel arrêté), il faut un mécanisme permettant à l'ensemble de ces participants de **s'accorder sur un état unique et cohérent du registre** — c'est-à-dire, concrètement, sur quel bloc suit quel bloc — tout en résistant à des tentatives délibérées de manipulation. Ce problème n'est pas propre aux blockchains : c'est une instance directe d'un problème classique et bien étudié en informatique distribuée, le **problème des généraux byzantins**, qui formalise la difficulté d'obtenir un accord fiable entre participants distants lorsque certains d'entre eux peuvent transmettre des informations contradictoires ou mensongères. C'est précisément ce que résolvent, chacun avec des compromis différents, les **mécanismes de consensus** présentés dans ce chapitre.

2. Preuve de travail (Proof of Work, PoW)

La **preuve de travail** est le mécanisme de consensus historique, popularisé par Bitcoin. Son principe est le suivant : pour avoir le droit de proposer un nouveau bloc, un participant du réseau — appelé **mineur** — doit résoudre un **défi calculatoire coûteux**. Concrètement, il doit trouver une valeur particulière, appelée **nonce**, telle que l'empreinte de hachage de l'en-tête du bloc (incluant ce nonce) satisfasse une condition de difficulté arbitrairement exigeante, par exemple : commencer par un certain nombre de zéros en écriture binaire ou hexadécimale.

Il n'existe aucun raccourci mathématique connu pour trouver directement un tel nonce : en pratique, un mineur doit essayer un très grand nombre de valeurs différentes, une par une, jusqu'à en trouver une qui fonctionne (« essai-erreur » systématique, aussi appelé recherche par force brute sur l'espace des nonces). Ce calcul est donc **intentionnellement coûteux à réaliser**, en temps de calcul et en électricité consommée, mais en contrepartie **trivial à vérifier** par n'importe quel autre nœud du réseau : il suffit de recalculer une seule fois l'empreinte de hachage du bloc proposé et de constater qu'elle satisfait bien la condition de difficulté annoncée.

- **Sécurité** : un attaquant souhaitant falsifier l'historique — par exemple pour effectuer une double dépense — devrait disposer d'une puissance de calcul supérieure à celle de l'ensemble du reste du réseau honnête combiné, afin de pouvoir recalculer plus rapidement une chaîne alternative de blocs. C'est ce qu'on appelle une **attaque des 51 %** : un coût économique considérable et dissuasif dès lors que le réseau est suffisamment décentralisé et que sa puissance de calcul cumulée est importante.
- **Limite majeure** : la preuve de travail implique, par construction, une **consommation énergétique très importante**, puisque l'ensemble des mineurs en compétition dépensent continuellement de l'énergie électrique pour tenter de résoudre le défi calculatoire avant les autres, y compris les mineurs qui ne trouveront finalement pas le bon nonce. C'est cette limite structurelle qui a motivé la recherche active d'alternatives moins gourmandes en ressources.

3. Preuve d'enjeu (Proof of Stake, PoS)

La **preuve d'enjeu** propose une approche radicalement différente pour attribuer le droit de proposer et de valider un bloc : plutôt que de le faire dépendre d'une puissance de calcul consommée en continu, ce droit est attribué — souvent par un tirage aléatoire pondéré — en fonction de la **quantité de cryptomonnaie qu'un participant a mise en jeu** (*staked*), c'est-à-dire immobilisée et engagée comme garantie de bonne conduite sur le réseau.

- **Sécurité** : le mécanisme d'incitation change de nature par rapport à la preuve de travail. Un participant (**validateur**) qui tenterait de valider des blocs contradictoires, invalides, ou de tricher d'une autre manière détectable par le protocole, risque de perdre (« **slashing** ») une partie ou la totalité de sa mise engagée. Le comportement honnête devient ainsi la stratégie économiquement rationnelle : l'intérêt financier du validateur est directement aligné avec l'intérêt du réseau dans son ensemble.
- **Avantage** : la consommation énergétique de la preuve d'enjeu est très inférieure à celle de la preuve de travail, puisqu'elle ne repose plus sur une compétition de calcul continue entre participants. Ethereum est passé de la preuve de travail à la preuve d'enjeu en septembre 2022 (un changement connu sous le nom de « The Merge »), réduisant considérablement l'empreinte énergétique du réseau.
- **Débat en cours** : la preuve d'enjeu soulève un débat non tranché sur le risque de **centralisation accrue** au profit des plus gros détenteurs de la cryptomonnaie concernée. Le raisonnement critique s'exprime souvent ainsi : plus un participant détient de capital à mettre en jeu, plus il a de chances d'être sélectionné pour valider des blocs et percevoir les récompenses associées, ce qui accroît encore son capital et donc son influence future sur le réseau — un mécanisme potentiellement auto-renforçant que les concepteurs de protocoles PoS cherchent à atténuer par diverses règles (plafonds, sélection aléatoire pondérée non strictement linéaire, etc.).

4. Tolérance byzantine pratique (PBFT et variantes)

Une troisième famille de mécanismes, la **tolérance byzantine pratique** (*Practical Byzantine Fault Tolerance*, PBFT) et ses nombreuses variantes, est typiquement utilisée dans les **blockchains à permission**, où le nombre de participants habilités à valider est limité et connu à l'avance (chapitre 1).

Le principe diffère fondamentalement de PoW et PoS : les nœuds valident un bloc par un **mécanisme de vote explicite** entre eux, plutôt que par un calcul coûteux ou une mise en jeu de capital. Le protocole tolère qu'une fraction des participants — typiquement jusqu'à un tiers du total — soit défaillante ou activement malveillante, tout en garantissant que les nœuds honnêtes restants parviennent malgré tout à un accord cohérent. Un avantage pratique important de cette approche est sa **finalité quasi immédiate** : dès qu'un nombre suffisant de votes valides est réuni, le bloc est considéré comme définitivement validé, contrairement à la preuve de travail où la certitude qu'un bloc ne sera pas remis en cause augmente seulement **progressivement** avec le nombre de confirmations ultérieures.

5. Comparaison synthétique

Les trois familles de mécanismes présentées reposent sur des logiques de sécurité fondamentalement différentes, résumées ci-dessous :

Mécanisme	Coût de sécurité	Consommation énergétique	Finalité	Exemple
PoW	puissance de calcul	très élevée	probabiliste (croît avec les confirmations)	Bitcoin
PoS	capital économique en jeu	faible	souvent quasi immédiate selon l'implémentation	Ethereum (depuis 2022)
BFT	identité/réputation des validateurs	faible	immédiate	réseaux à permission, certaines blockchains d'entreprise

Ce tableau ne doit pas être lu comme un classement de qualité : chaque mécanisme représente un compromis différent entre décentralisation réelle (nombre et diversité des participants qui peuvent effectivement contribuer à la validation), sécurité économique, performance et consommation de ressources. Le choix pertinent dépend du contexte d'usage visé, un point qui sera repris au chapitre 6.

6. Fork et résolution de conflits

Même avec un mécanisme de consensus bien conçu, il arrive qu'à un instant donné, **deux blocs valides** soient proposés presque simultanément par deux participants différents — une situation tout à fait normale dans un réseau distribué où l'information met un certain temps à se propager entre nœuds géographiquement dispersés. Cette situation crée temporairement une **fourche** (*fork*) : deux versions concurrentes de la chaîne coexistent brièvement.

Le protocole définit une règle claire pour résoudre cette situation : les nœuds convergent généralement vers la **chaîne la plus longue** (règle typique de la preuve de travail, où la longueur reflète la quantité de travail calculatoire cumulée) ou vers celle **validée par le plus grand nombre de participants** (règle plus courante en preuve d'enjeu). Une fois la convergence effectuée, les blocs de la chaîne écartée sont abandonnés et leurs transactions, si elles n'ont pas été incluses dans la chaîne retenue, redeviennent en attente de validation.

Il faut distinguer ces fourches temporaires et involontaires d'un **hard fork** délibéré : un changement de règles du protocole incompatible avec les versions précédentes, décidé lors d'une mise à jour majeure du logiciel ou à la suite d'un désaccord de gouvernance profond au sein de la communauté des développeurs et des utilisateurs. Un hard fork peut, dans les cas les plus conflictuels, aboutir à la coexistence durable de deux réseaux distincts issus d'une même histoire commune.

À retenir

- Le mécanisme de consensus répond à un problème classique d'informatique distribuée (accord dans un réseau potentiellement malveillant, sans autorité centrale), analogue au problème des généraux byzantins.
- PoW sécurise par le coût calculatoire mais consomme beaucoup d'énergie ; PoS sécurise par un risque économique direct (slashing) avec une empreinte énergétique bien moindre ; BFT offre une finalité immédiate mais suppose des validateurs identifiés.
- Le choix du mécanisme de consensus détermine directement les propriétés de sécurité, de décentralisation réelle et de coût du réseau — un compromis, pas une hiérarchie absolue de qualité.
- Les fourches temporaires sont un phénomène normal résolu par une règle de convergence du protocole ; un hard fork est un changement délibéré et incompatible des règles elles-mêmes.

Chapitre 4 — Smart contracts et Ethereum

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Qu'est-ce qu'un smart contract ?

Un **smart contract** (« contrat intelligent ») est un programme informatique **déployé directement sur une blockchain**, dont le code et l'état s'exécutent de façon **déterministe** — c'est-à-dire que, pour une même entrée et un même état de départ, tous les nœuds du réseau qui exécutent ce programme obtiennent exactement le même résultat — et **vérifiable** par l'ensemble de ces nœuds, chacun réexécutant indépendamment le même calcul pour s'assurer de sa validité.

Une fois déployé, le code d'un smart contract ne peut en principe plus être modifié unilatéralement par son auteur d'origine, sauf si un mécanisme d'évolutivité (*upgradeability*) a été explicitement prévu dès la conception du contrat — un choix de conception qui a lui-même des implications de sécurité importantes, discutées au chapitre suivant.

Le terme « contrat » est trompeur si on l'entend au sens juridique classique : un smart contract n'est pas, par lui-même, un contrat légalement contraignant reconnu par un système judiciaire. Il s'agit avant tout d'un **programme auto-exécutable** : une fois les conditions codées dans le programme réunies, l'action correspondante (transfert de fonds, mise à jour d'un état) s'exécute automatiquement, sans intervention humaine ni possibilité de négociation a posteriori. La portée juridique effective d'un smart contract — sa reconnaissance comme preuve d'un accord entre parties, son opposabilité devant un tribunal — dépend entièrement du contexte réglementaire de la juridiction concernée, qui varie considérablement d'un pays à l'autre et continue d'évoluer.

2. Ethereum : une blockchain programmable

Bitcoin a été conçu essentiellement pour transférer une unité de valeur d'une adresse à une autre, avec un langage de script volontairement limité pour des raisons de sécurité et de prévisibilité. **Ethereum**, lancé en 2015, a étendu ce modèle de façon décisive en introduisant une **machine virtuelle Turing-complète** : l'**EVM** (*Ethereum Virtual Machine*). Une machine Turing-complète peut, en principe, exécuter n'importe quel programme calculable — boucles, conditions, appels de fonctions imbriqués — ce qui permet d'exécuter sur la blockchain des programmes arbitrairement complexes, et non plus seulement de simples transactions de transfert de valeur.

Notion de « gas »

Donner à une blockchain publique la capacité d'exécuter des programmes Turing-complets pose un problème classique en informatique théorique : le **problème de l'arrêt** — il n'existe pas, en général, de méthode permettant de prédire à l'avance si un programme donné se terminera ou tournera indéfiniment. Dans un contexte adversarial, où n'importe qui peut soumettre un programme à exécuter par l'ensemble du réseau, cela ouvrirait la porte à des boucles infinies délibérément malveillantes, capables de paralyser le réseau tout entier.

Ethereum répond à ce problème par le mécanisme du **gas** : chaque opération élémentaire exécutée par l'EVM (une addition, un accès en mémoire, un appel à un autre contrat, etc.) a un coût fixé en unités de gas. L'émetteur d'une transaction doit payer ce gas, dans la cryptomonnaie native du réseau (l'**Ether**), et doit fixer à l'avance une limite maximale de gas qu'il accepte de dépenser pour cette transaction. Ce mécanisme remplit un double rôle : il **rémunère les validateurs** pour les ressources de calcul et de stockage effectivement consommées par l'exécution du contrat, et il **empêche les boucles infinies ou les attaques par déni de service**, puisqu'un programme épuise inévitablement son budget de gas avant de pouvoir consommer des ressources indéfiniment — une transaction dont le gas s'épuise est simplement annulée (avec perte du gas déjà consommé), sans que le réseau n'ait à décider si le programme se serait un jour arrêté. C'est une solution pratique et élégante, propre au contexte adversarial d'une blockchain publique, au problème théorique de l'arrêt.

3. Langages et environnement de développement

L'écriture, le test et le déploiement de smart contracts s'appuient aujourd'hui sur un écosystème d'outils relativement mature :

- **Solidity** est le langage le plus utilisé pour écrire des smart contracts sur Ethereum et sur les nombreuses blockchains compatibles avec l'EVM. Sa syntaxe, volontairement proche de celle du JavaScript et du C++, facilite sa prise en main par des développeurs déjà familiers avec ces langages, tout en intégrant des concepts propres au contexte blockchain (gestion explicite du gas, types adaptés aux montants de cryptomonnaie, mécanismes de contrôle d'accès).
- Plusieurs **environnements de développement et de test** facilitent le cycle de développement complet : **Remix** est un environnement de développement intégré (IDE) accessible directement dans un navigateur web, particulièrement adapté à l'apprentissage et au prototypage rapide ; **Hardhat** et **Foundry** sont des environnements plus complets, orientés vers des projets de plus grande envergure, offrant des fonctionnalités avancées de test automatisé, de simulation et de déploiement scriptable.
- Les **réseaux de test** (*testnets*) sont des répliques du réseau principal (*mainnet*) fonctionnant selon les mêmes règles de protocole, mais utilisant une cryptomonnaie sans valeur financière réelle. Ils sont indispensables pour tester un contrat en conditions réalistes — y compris son interaction avec d'autres contrats déjà déployés — avant tout déploiement sur le réseau principal, où une erreur peut avoir des conséquences financières irréversibles une fois le contrat déployé. C'est sur ce type de réseau que se dérouleront les travaux pratiques de ce module (TP3/TP4).

4. Exemple simplifié de structure d'un smart contract (Solidity)

L'exemple suivant illustre un contrat volontairement minimal, permettant à n'importe quel utilisateur de déposer et de retirer de l'Ether auprès du contrat, un peu à la manière d'un coffre-fort numérique individuel :

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract CompteSimple {
    mapping(address => uint256) public soldes;

    function deposer() external payable {
        soldes[msg.sender] += msg.value;
    }

    function retirer(uint256 montant) external {
        require(soldes[msg.sender] >= montant, "Solde insuffisant");
        soldes[msg.sender] -= montant;
        payable(msg.sender).transfer(montant);
    }
}
```

Ce court exemple illustre déjà plusieurs notions clés propres à la programmation de smart contracts : un **état persistant** sur la blockchain, stocké ici dans un `mapping` associant chaque adresse à son solde et conservé de façon durable entre les appels ; des **vérifications explicites** effectuées avec `require`, qui annule intégralement la transaction (et restitue le gas non consommé) si la condition n'est pas satisfaite ; et la **manipulation directe de valeur** (Ether), via les mots-clés `payable` qui autorisent une fonction à recevoir ou envoyer des fonds.

Cet exemple prépare directement le chapitre suivant sur la sécurité : l'**ordre des opérations** dans la fonction `retirer` — vérifier le solde, puis le décrémenter, puis seulement transférer les fonds — n'est pas un détail de style, mais une décision de sécurité déterminante, comme le montrera l'étude de la vulnérabilité de réentrance.

5. Cas d'usage des smart contracts

Au-delà de l'exemple pédagogique ci-dessus, les smart contracts servent de fondation à un ensemble de cas d'usage désormais bien établis dans l'écosystème Ethereum et les blockchains compatibles :

- **Jetons (tokens)** : les smart contracts permettent de représenter des actifs numériques selon des **standards** largement adoptés par l'écosystème, ce qui garantit leur interopérabilité entre applications. Le standard **ERC-20** définit une interface commune pour des jetons **fongibles** (interchangeables entre eux, comme une monnaie), tandis que le standard **ERC-721** définit une interface pour des jetons **non fongibles** (NFT), chacun étant unique et distinct des autres.
- **Finance décentralisée (DeFi)** : des applications de prêt, d'échange décentralisé (*DEX — Decentralized Exchange*) et d'autres produits financiers fonctionnent entièrement via des smart contracts, sans intermédiaire financier traditionnel (banque, courtier) — les règles du service (taux d'intérêt, conditions d'échange) sont directement codées dans le contrat.
- **Gouvernance décentralisée (DAO)** : une *Decentralized Autonomous Organization* utilise des smart contracts pour organiser des votes entre membres et exécuter automatiquement les décisions adoptées selon des règles définies à l'avance dans le code, sans organe de direction centralisé au sens classique.
- **Traçabilité** : des smart contracts peuvent enregistrer et faire respecter des étapes de suivi d'une chaîne logistique ou de certification de provenance d'un produit, en s'appuyant sur l'intégrité garantie par la blockchain sous-jacente (avec, ici encore, la limite du problème de l'oracle décrite ci-dessous).

6. Le problème de l'oracle

Un smart contract, par conception, ne peut accéder qu'aux **données déjà présentes sur la blockchain** au moment de son exécution : il n'a aucun moyen direct de consulter une donnée du monde extérieur — le cours actuel d'une action en bourse, le résultat d'un match sportif, la température relevée par un capteur météo, ou tout autre fait du monde réel. Cette limitation découle directement de l'exigence de **déterminisme** évoquée en début de chapitre : si un contrat pouvait interroger librement une source externe, deux nœuds exécutant le même contrat au même moment pourraient obtenir des réponses différentes (la donnée externe ayant pu changer entre les deux requêtes), brisant l'accord de l'ensemble du réseau sur le résultat de l'exécution.

Un **oracle** est un service tiers chargé d'apporter cette donnée externe sur la blockchain, généralement en la publiant lui-même dans une transaction que le smart contract pourra ensuite lire comme n'importe quelle autre donnée déjà présente sur la chaîne. Ce mécanisme, s'il résout le problème d'accès pratique, introduit un **point de risque majeur** : la sécurité et la correction du smart contract tout entier dépendent désormais de la **fiabilité de l'oracle** utilisé. Si l'oracle transmet une donnée erronée — par erreur ou par manipulation délibérée — le smart contract l'exécutera fidèlement comme s'il s'agissait d'une vérité incontestable, puisqu'il n'a aucun moyen de la questionner. L'oracle redevient ainsi, de fait, un **point de confiance centralisé**, en tension directe avec l'objectif initial de décentralisation qui motive l'usage même d'une blockchain. Cette tension sera reprise au chapitre 5 (manipulation d'oracle de prix) et au chapitre 6 (limites pratiques des applications blockchain).

À retenir

- Un smart contract est un programme auto-exécutable, déterministe et vérifiable par tous les nœuds, pas nécessairement un contrat juridique au sens classique.
- Le mécanisme de gas rémunère l'exécution et empêche les boucles infinies en apportant une solution pratique au problème théorique de l'arrêt, brique essentielle de la sécurité opérationnelle d'Ethereum.
- Solidity, associé à des environnements comme Remix, Hardhat ou Foundry et à des réseaux de test, constitue l'écosystème standard de développement de smart contracts.
- Le problème de l'oracle rappelle qu'un smart contract ne résout la confiance que pour les données déjà présentes sur la chaîne, pas pour les données du monde réel qui y sont importées.

Chapitre 5 — Sécurité et vulnérabilités des smart contracts

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Pourquoi la sécurité des smart contracts est particulièrement critique

La sécurité applicative classique repose largement sur la possibilité de **corriger une faille après coup** : un correctif est développé, testé, puis déployé discrètement en production, souvent sans que les utilisateurs n'en aient même conscience. Ce modèle ne s'applique pas, ou très difficilement, aux smart contracts déployés sur une blockchain publique.

Un smart contract déployé est, par construction, **immuable** (chapitre 4) et son **code est public** : n'importe qui, y compris un attaquant potentiel, peut lire le code exact déployé sur la blockchain et l'analyser à loisir, hors ligne et sans limite de temps, pour y chercher une faille. Contrairement à une application web classique, il n'est généralement pas possible de corriger discrètement une faille découverte après déploiement : le code vulnérable reste actif, exécutable et visible de tous, jusqu'à ce qu'une migration complexe vers un nouveau contrat soit organisée — ce qui suppose souvent de convaincre l'ensemble des utilisateurs de migrer volontairement leurs fonds ou leurs interactions vers la nouvelle version, une opération loin d'être triviale ni instantanée.

À cette difficulté structurelle s'ajoute un facteur aggravant : de nombreux smart contracts, en particulier dans le domaine de la finance décentralisée (chapitre 4), manipulent **directement des fonds réels**, parfois des montants considérables agrégés dans un seul contrat. Cela en fait des cibles à **forte valeur immédiate** pour un attaquant : contrairement à une fuite de données qui doit ensuite être monétisée, l'exploitation réussie d'une faille dans un smart contract financier peut se traduire en gain immédiat et directement transférable. Cette combinaison — immutabilité, transparence du code, et valeur financière directement en jeu — explique pourquoi l'audit préalable au déploiement occupe une place centrale dans le développement de smart contracts, bien plus que dans le développement logiciel classique.

2. Réentrance (reentrancy)

La **réentrance** est une vulnérabilité historique majeure du développement de smart contracts, largement étudiée dans la littérature de sécurité depuis l'incident public dit « The DAO », survenu en 2016 sur le réseau Ethereum et qui a marqué durablement la prise de conscience de la communauté quant à l'importance de l'audit des smart contracts.

Le principe de cette vulnérabilité repose sur un décalage temporel dangereux entre deux opérations : un contrat appelle un **contrat externe** (par exemple pour transférer des fonds à un bénéficiaire) **avant** d'avoir mis à jour son propre état interne (par exemple, avant d'avoir décrémenté le solde correspondant). Si le contrat externe appelé est malveillant, il peut alors **rappeler la fonction d'origine de façon récursive**, depuis l'intérieur même de l'appel externe en cours, exploitant précisément le fait que l'état interne (le solde) n'a pas encore été mis à jour au moment de ce rappel — le contrat vulnérable « croit » donc, à chaque appel récursif, que le solde est toujours disponible.

```
// Version vulnérable
function retirer(uint256 montant) external {
    require(soldes[msg.sender] >= montant, "Solde insuffisant");
    payable(msg.sender).transfer(montant); // appel externe AVANT la mise à jour de l'état
    soldes[msg.sender] -= montant;         // trop tard : un contrat malveillant a pu rappeler retirer() entre-temps
}
```

Contre-mesure : la défense de référence contre cette classe de vulnérabilité est le patron de conception dit **checks-effects-interactions** (« vérifications, puis effets, puis interactions ») : on effectue d'abord toutes les **vérifications** nécessaires (`require`), puis on met à jour l'**état interne** du contrat, et ce n'est **qu'ensuite** que l'on procède à l'**interaction externe** (appel à un autre contrat, transfert de fonds). C'est exactement l'ordre correct déjà appliqué dans l'exemple de la fonction `retirer` présenté au chapitre 4 — un ordre qui, à première vue, peut sembler un simple détail stylistique, mais qui constitue en réalité une décision de sécurité déterminante.

3. Débordement/souppassement d'entier (overflow/underflow)

Historiquement, une variable numérique non signée en Solidity (par exemple de type `uint256`) pouvait « déborder » silencieusement lorsqu'une opération arithmétique dépassait ses bornes représentables : une addition dépassant la valeur maximale représentable **revenait à zéro** (*overflow*), et une soustraction produisant un résultat négatif à partir d'un type non signé revenait à la **valeur maximale représentable** (*underflow*), sans qu'aucune erreur ne soit levée par défaut. Cette absence de vérification automatique a été exploitée pour créer artificiellement des soldes ou des quantités de jetons énormes à partir d'opérations arithmétiques détournées de leur usage prévu.

Depuis la version 0.8 de Solidity, ces vérifications sont effectuées **automatiquement par défaut** : le compilateur insère des contrôles qui **annulent la transaction** dès qu'un débordement ou un souppassement se produirait, éliminant la classe de vulnérabilité pour tout contrat compilé avec une version récente utilisant le comportement par défaut. Le risque reste cependant réel dans deux cas : du code écrit et déployé avec des **versions antérieures** du compilateur, encore en circulation sur certains réseaux, et l'usage explicite de blocs `unchecked` — une fonctionnalité permettant de désactiver volontairement ces vérifications automatiques pour des raisons d'optimisation de gas — lorsque cette désactivation n'est pas soigneusement justifiée et bornée par ailleurs.

4. Contrôle d'accès défaillant

Une fonction sensible d'un smart contract — par exemple, une fonction permettant de modifier le propriétaire du contrat, de retirer des fonds réservés à un administrateur, ou de mettre à jour un paramètre critique du système — doit systématiquement vérifier que l'appelant dispose bien des **droits appropriés** avant d'exécuter l'opération, typiquement via une condition telle que `require(msg.sender == proprietaire)`. L'absence ou l'insuffisance de cette vérification permet à **n'importe quel utilisateur** du réseau, y compris un attaquant n'ayant aucun droit légitime, d'appeler directement cette fonction sensible et d'en déclencher les effets.

Cette classe de vulnérabilité est l'équivalent, dans le monde des smart contracts, du **contrôle d'accès défaillant**, classé en tête du classement OWASP Top 10 des risques applicatifs les plus critiques (module *Audit organisation et technique*) : il s'agit d'un problème transversal, présent dans de nombreuses catégories de logiciels, dont les smart contracts ne sont qu'une déclinaison particulière — avec la circonstance aggravante, propre à ce contexte, que l'immutabilité du contrat déployé empêche une correction rapide une fois la faille découverte.

5. Attaques par manipulation d'oracle de prix

Cette vulnérabilité prolonge directement le **problème de l'oracle** introduit au chapitre 4. Un contrat de finance décentralisée qui détermine le prix d'un actif en s'appuyant sur une source de données **manipulable** — par exemple la liquidité instantanée disponible sur un seul échange décentralisé, à un instant précis — peut être trompé par un attaquant capable de faire varier temporairement ce prix perçu par le contrat.

Une technique fréquemment associée à ce type d'attaque est le **prêt flash** (*flash loan*) : un emprunt de fonds sans garantie préalable, autorisé à condition d'être intégralement remboursé — capital et frais éventuels — **au sein de la même transaction blockchain** qui l'a initié. En combinant un prêt flash avec une manipulation ciblée d'un marché de faible liquidité, un attaquant peut faire temporairement varier le prix observé par l'oracle du contrat visé, le temps d'une seule transaction, afin de déclencher des conditions artificiellement favorables dans ce contrat (emprunt sous-garanti, échange à un taux anormal), avant de rembourser le prêt flash et de repartir avec le gain ainsi obtenu. La contre-mesure de conception la plus robuste consiste à s'appuyer sur des **oracles décentralisés et résistants à la manipulation** (agrégeant plusieurs sources indépendantes, avec des mécanismes de lissage temporel), plutôt que sur une source de prix unique et instantanée.

6. Attaques par déni de service et boucles coûteuses en gas

Une fonction d'un smart contract qui **itère** (parcourt en boucle) une structure de données dont la **taille peut être influencée par un attaquant** — par exemple une liste d'utilisateurs inscrits, que n'importe qui peut faire croître en s'y ajoutant — présente un risque structurel : si cette liste devient suffisamment grande, l'exécution complète de la boucle peut consommer **plus de gas que le maximum autorisé par bloc** (chapitre 4). La fonction devient alors **définitivement inexécutable** : chaque tentative d'appel échoue systématiquement par épuisement du gas disponible, avant même d'atteindre la fin de la boucle. C'est un **déni de service permanent**, non corrigible sans une migration complète du contrat vers une nouvelle version conçue pour éviter ce type de dépendance entre la taille d'une structure de données et le coût d'exécution d'une fonction.

7. Démarche d'audit d'un smart contract

Compte tenu de l'immutabilité des contrats déployés et de l'enjeu financier souvent direct, la sécurisation d'un smart contract repose sur une démarche d'audit **préalable au déploiement**, structurée en plusieurs étapes complémentaires plutôt que sur une seule technique isolée :

1. **Revue manuelle du code**, en confrontant systématiquement chaque fonction sensible aux classes de vulnérabilités présentées dans ce chapitre — une lecture ligne par ligne, guidée par une check-list de risques connus.
2. **Analyse statique automatisée**, à l'aide d'outils dédiés (par exemple Slither ou MythX) capables de détecter automatiquement des motifs de code correspondant à des vulnérabilités connues, sans avoir besoin d'exécuter réellement le contrat.
3. **Tests unitaires et de fuzzing** : les tests unitaires vérifient le comportement attendu du contrat sur des scénarios précis définis par le développeur, tandis que le fuzzing génère automatiquement un grand nombre d'entrées, y compris des cas limites inattendus, afin de détecter des comportements anormaux non anticipés par l'équipe de développement.
4. **Vérification formelle**, pour les contrats à très haute valeur financière ou à criticité particulièrement élevée : une démarche consistant à établir une **preuve mathématique** que le contrat respecte certaines propriétés de sécurité définies formellement, une garantie bien plus forte que le simple test, mais aussi bien plus coûteuse à mettre en œuvre.
5. **Programme de récompense pour la découverte de vulnérabilités** (*bug bounty*), avant et après déploiement : inviter des chercheurs en sécurité externes à rechercher activement des failles en échange d'une récompense, dans un cadre légal et éthique défini à l'avance, plutôt que de laisser cette découverte au hasard ou à un attaquant malveillant.

À retenir

- La réentrance, le contrôle d'accès défaillant et la manipulation d'oracle sont des classes de vulnérabilités récurrentes des smart contracts, ayant historiquement conduit à des pertes financières réelles considérables.
- L'immutabilité des smart contracts déployés rend la prévention (audit avant déploiement) bien plus critique que pour une application web classique corrigible a posteriori.
- Le patron checks-effects-interactions est la contre-mesure de référence contre la réentrance, illustrant l'importance de l'ordre des opérations dans le code.
- Une démarche d'audit sérieuse combine revue manuelle, analyse statique, tests/fuzzing, et pour les contrats à haute valeur, vérification formelle et bug bounty.

Chapitre 6 — Applications, enjeux et limites

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Panorama des applications au-delà des cryptomonnaies

Si les cryptomonnaies (Bitcoin, Ether) restent l'application la plus visible et la plus ancienne de la technologie blockchain, les propriétés étudiées dans les chapitres précédents — registre partagé sans tiers de confiance unique, intégrité vérifiable, exécution automatisée via les smart contracts — ont trouvé des applications dans des domaines assez différents :

- **Finance décentralisée (DeFi)** : prêts, échanges décentralisés, produits dérivés, fonctionnant sans intermédiaire financier centralisé, en s'appuyant directement sur les mécanismes de smart contracts étudiés au chapitre 4. Ce domaine reste exposé aux risques de sécurité détaillés au chapitre 5, ainsi qu'à une volatilité de marché et une situation réglementaire encore incertaines selon les juridictions.
- **Traçabilité de chaîne logistique** : suivi de la provenance de produits (agroalimentaire, textile, minerais) tout au long d'une chaîne d'approvisionnement, où la blockchain garantit l'**intégrité** des enregistrements une fois qu'ils y ont été saisis — personne ne peut discrètement modifier un enregistrement passé — sans pour autant garantir la **véracité** de la saisie initiale (problème de l'oracle, rappel du chapitre 4) : si un maillon de la chaîne saisit une information fausse dès le départ, la blockchain la conservera fidèlement, mais fausse.
- **Identité numérique décentralisée (self-sovereign identity)** : une approche visant à permettre à un individu de **contrôler lui-même** ses données d'identité et de **prouver sélectivement** certains attributs (par exemple, prouver qu'il est majeur sans révéler sa date de naissance exacte) sans dépendre d'un registre central unique détenu par un État ou une entreprise.
- **Certification et notariation** : preuve d'existence et d'horodatage d'un document à une date donnée, en ancrant l'empreinte de hachage de ce document sur une blockchain publique (sans jamais y publier le contenu du document lui-même). C'est un usage relativement simple et robuste de la blockchain, puisqu'il ne manipule aucune valeur financière et se limite à exploiter la propriété d'intégrité horodatée du registre.
- **Vote électronique** : un cas d'usage souvent évoqué dans les discours de vulgarisation, mais particulièrement délicat à réaliser correctement en pratique, car il combine des exigences difficilement conciliables entre elles : l'**anonymat de l'électeur** (personne ne doit pouvoir savoir pour qui il a voté), la **vérifiabilité publique du résultat** (chacun doit pouvoir vérifier que son vote a bien été comptabilisé et que le résultat global est exact), et la **résistance à la coercition** (un électeur ne doit pas pouvoir prouver a posteriori à un tiers pour qui il a voté, afin d'éviter l'achat ou la contrainte de votes). Ce domaine reste, à ce jour, davantage un objet de recherche active qu'un terrain de déploiements matures et largement adoptés à grande échelle.

2. Trilemme de la blockchain (scalabilité, sécurité, décentralisation)

Les concepteurs de blockchains publiques font face à un compromis largement reconnu dans la communauté, souvent désigné sous le nom de **trilemme de la blockchain** : il est difficile d'optimiser simultanément trois propriétés à la fois — la **scalabilité** (le nombre de transactions que le réseau peut traiter par seconde), la **sécurité** (la résistance du réseau aux tentatives d'attaque ou de falsification) et la **décentralisation** (le nombre et l'indépendance réelle des participants qui contribuent à la validation).

En pratique, les architectures existantes tendent à privilégier deux de ces trois propriétés, au détriment relatif de la troisième. Bitcoin, par exemple, privilégie fortement la sécurité et la décentralisation, au prix d'une scalabilité limitée (un nombre de transactions par seconde relativement faible comparé à un système de paiement centralisé). À l'inverse, certaines blockchains à permission (chapitre 1), en limitant le nombre de validateurs à un groupe restreint et identifié, peuvent atteindre une scalabilité et une sécurité opérationnelle élevées, mais au prix d'une décentralisation nettement réduite.

Approches de mise à l'échelle (aperçu)

Plusieurs pistes techniques sont explorées ou déployées pour tenter d'assouplir ce compromis, sans prétendre l'éliminer totalement :

- **Solutions de couche 2 (Layer 2)** : l'idée générale consiste à traiter la **majorité des transactions en dehors de la chaîne principale** (*off-chain*), dans un environnement plus rapide et moins coûteux, tout en n'ancrant sur la chaîne principale qu'un **résumé périodique** de ces opérations — suffisant pour hériter des garanties de sécurité de la chaîne principale sans y faire transiter chaque transaction individuellement. Les *rollups* déployés sur Ethereum en sont un exemple représentatif de cette famille d'approches.
- **Sharding** : cette approche consiste à **répartir l'état et le traitement** du réseau entre plusieurs sous-réseaux, appelés « shards » (fragments), chacun traité en parallèle par un sous-ensemble de validateurs plutôt que par l'intégralité du réseau. L'objectif est d'augmenter la capacité globale de traitement du système en évitant que chaque nœud n'ait à valider absolument toutes les transactions du réseau entier.

3. Enjeux réglementaires

L'adoption croissante des cryptoactifs et des applications blockchain soulève des enjeux réglementaires substantiels, encore loin d'être stabilisés :

- **Lutte contre le blanchiment (AML) et connaissance du client (KYC)** : il existe une tension structurelle entre le **pseudonymat** propre aux blockchains publiques — une adresse n'est pas directement associée à une identité civile — et les obligations réglementaires imposées par de nombreuses juridictions aux plateformes d'échange de cryptoactifs, qui doivent généralement identifier leurs utilisateurs et surveiller certains mouvements de fonds suspects.
- **Qualification juridique des cryptoactifs** : selon les juridictions, un même jeton peut être qualifié de façon très différente — comme une monnaie, une valeur mobilière, ou un actif numérique *sui generis* (d'une catégorie propre, ne correspondant à aucune catégorie juridique préexistante) — avec des conséquences fiscales et réglementaires très différentes selon la qualification retenue.
- **Protection des consommateurs** : les utilisateurs de cryptoactifs s'exposent à une **volatilité** de marché souvent importante et à un **risque de perte totale et irréversible** des fonds, que ce soit en cas d'erreur de manipulation (perte définitive d'une clé privée, sans mécanisme de récupération) ou d'escroquerie, en l'absence de recours équivalent à ceux offerts par un système bancaire traditionnel régulé.
- **Encadrement progressif** : plusieurs juridictions ont mis en place, ou sont en cours de développement, des cadres réglementaires dédiés spécifiquement aux actifs numériques et aux prestataires de services associés. La situation évolue rapidement et diffère fortement d'un pays à l'autre, ce qui impose une vigilance particulière pour tout projet ou toute organisation opérant dans plusieurs juridictions à la fois.

4. Limites techniques et environnementales

Au-delà des enjeux réglementaires, plusieurs limites d'ordre technique et environnemental doivent être prises en compte lors de l'évaluation d'un projet blockchain :

- La **consommation énergétique** reste significative pour les réseaux fonctionnant en preuve de travail (chapitre 3), bien que la transition de certains réseaux majeurs (comme Ethereum) vers la preuve d'enjeu ait fortement réduit cet impact pour les réseaux concernés — cette réduction ne s'applique cependant pas aux réseaux qui conservent la preuve de travail, à commencer par Bitcoin.
- L'**irréversibilité des transactions** est à double tranchant : c'est un avantage recherché en termes de finalité et de résistance à la censure, mais c'est aussi un risque important en cas d'erreur humaine (envoi à la mauvaise adresse) ou de vol de clés privées — il n'existe, par conception, aucun « bouton d'annulation » ni service client capable d'inverser une transaction déjà confirmée.
- La **complexité d'usage** pour l'utilisateur final constitue un frein réel à l'adoption grand public : la gestion de clés privées (leur génération sécurisée, leur sauvegarde, leur protection contre le vol ou la perte) demande une rigueur et une compréhension technique que l'utilisateur moyen d'un service numérique classique n'a généralement pas à maîtriser, et l'absence de recours en cas de perte de clé aggrave encore les conséquences d'une erreur.

5. Regard critique : quand la blockchain est-elle pertinente ?

Face à l'engouement qui a entouré la technologie blockchain, il est utile de disposer d'une **grille de questions critiques** avant d'adopter cette solution pour un projet donné, plutôt que de l'adopter par simple effet de mode :

- A-t-on réellement besoin de **décentralisation**, ou une base de données classique administrée par un tiers de confiance suffirait-elle à répondre au besoin identifié ?
- Plusieurs parties **mutuellement méfiantes** doivent-elles effectivement partager un même registre, sans qu'aucune d'entre elles n'accepte de déléguer cette fonction à un intermédiaire commun ?
- L'**immutabilité** est-elle un avantage réel pour ce cas d'usage précis, ou au contraire une contrainte gênante — par exemple en tension avec un droit à l'effacement des données personnelles imposé par une réglementation applicable ?

Cette grille de questions, appliquée avec rigueur, conduit souvent à une conclusion inconfortable mais utile : de nombreux projets « blockchain » annoncés ces dernières années auraient pu être résolus de façon plus simple, plus rapide et moins coûteuse par une architecture centralisée classique, sans que le besoin réel de décentralisation n'ait été véritablement établi au préalable. Savoir reconnaître ces situations est une compétence professionnelle à part entière, aussi importante que la maîtrise technique de la blockchain elle-même.

À retenir

- Les applications de la blockchain dépassent les cryptomonnaies (DeFi, traçabilité, identité décentralisée, notariatisation, vote électronique) mais restent souvent confrontées au problème de l'oracle et au trilemme scalabilité/sécurité/décentralisation.
- Les solutions de couche 2 et le sharding sont des pistes de mise à l'échelle qui assouplissent le trilemme sans l'éliminer.
- Le cadre réglementaire des cryptoactifs (AML/KYC, qualification juridique, protection des consommateurs) est encore en construction et varie fortement selon les juridictions.
- La pertinence d'une solution blockchain doit être évaluée par rapport à un besoin réel de décentralisation, pas adoptée par principe.